Week 6 · Fri

JavaScript Form Validation

Full control over user input — beyond HTML5 limits

K0126002 · Gachon University · 2026 Fall · Dr. Toshpulatov

Why JavaScript Validation?

Capability	HTML5 only	JS Validation
Custom error messages	Generic browser tooltip	Your own styled below the input
Validation timing	Only on submit	As user types (input event) or on blur
Cross-field logic	Cannot compare two fields	Compare password + confirm password
Async checks	Not possible	Check username availability via fetch()
Styling errors	Browser-styled bubble	Full CSS control over error state

event.preventDefault()

```
// Add novalidate to <form> first!
// <form id="reg-form" novalidate>

const form = document.getElementById(
  "reg-form"
);

form.addEventListener("submit",
  function(event) {

    // Stop the browser reload
    event.preventDefault();

    if (validateForm()) {
      // All checks passed!
      form.submit();
    }
    // else: stay on page, show errors
});
```

Stop & validate before submit

`submit event`

Fires when form is submitted

`preventDefault()`

Stop the page from reloading

`validate()`

Our own check function

`return false`

Old way — avoid in modern JS

`novalidate`

Add to <form> to disable HTML5

QUICK QUIZ

What does event.preventDefault() do when called inside a form's submit handler?

A Deletes the form data

B Stops the form from reloading the page

C Validates all fields automatically

D Hides the form

Discuss with your neighbor — then we'll reveal the answer!

✅ ANSWER

What does event.preventDefault() do when called inside a form's submit handler?

A Deletes the form data

B Stops the form from reloading the page

C Validates all fields automatically

D Hides the form

It cancels the default browser action (page reload + HTTP request), letting your JS handle submission instead. Without it, the page reloads before your JS runs.

checkValidity() & setCustomValidity()

```
const sid = document.getElementById("sid");

// Check if input passes its HTML5 rules
if (!sid.checkValidity()) {
  console.log(sid.validationMessage);
}

// Set a CUSTOM error message
function validateSID() {
  const val = sid.value;
  if (!/^\d{9}$/.test(val)) {
    sid.setCustomValidity(
      "Student ID must be 9 digits"
    );
  } else {
    // IMPORTANT: clear the error
    sid.setCustomValidity("");
  }
}
```

Built-in JS validation API

`checkValidity()`

Returns true/false

`setCustomValidity()`

Set a custom error string

`setCustomValidity("")`

Clear error (mark valid)

`reportValidity()`

Show the browser tooltip

The validity Object

```
const email = document.getElementById("email");

// Read individual validity flags
function getEmailError(input) {
  const v = input.validity;
  if (v.valueMissing)
    return "Email is required";
  if (v.typeMismatch)
    return "Enter a valid email address";
  if (v.tooShort)
    return "Email is too short";
  return ""; // valid
}

// v.valid is true only when all
// flags above are false
```

Fine-grained error info

`valueMissing`

required but empty

`typeMismatch`

Wrong type (email, url)

`patternMismatch`

RegEx pattern failed

`tooShort/Long`

minlength / maxlength

`rangeUnderflow`

Below min value

QUICK QUIZ

You call field.setCustomValidity('') (empty string). What is the effect?

A Shows an error message

B Marks the field as valid / clears any custom error

C Deletes the field value

D Disables the field

Discuss with your neighbor — then we'll reveal the answer!

✅ ANSWER

You call field.setCustomValidity('') (empty string). What is the effect?

A Shows an error message

B Marks the field as valid / clears any custom error

C Deletes the field value

D Disables the field

An empty string resets the custom validity, making the field valid again. Any non-empty string sets an error message and marks the field invalid.

Real-time Feedback

```
<!-- HTML: error span below input -->
<input type="email" id="email">
<span id="email-error" class="error"
      aria-live="polite"></span>

// JS: show error as user types
const emailInput =
  document.getElementById("email");
const emailError =
  document.getElementById("email-error");

emailInput.addEventListener("input", () => {
  const msg = getEmailError(emailInput);
  emailError.textContent = msg;
  emailInput.setAttribute(
    "aria-invalid", msg ? "true" : "false"
  );
});
```

"input" event as user types

`"input" event`

Fires on every keystroke

`"blur" event`

Fires when focus leaves field

`"change" event`

Fires after value changes

`showError()`

Update the error

`clearError()`

Remove error on valid

QUICK QUIZ

Which event fires continuously AS the user types in an input field?

A keypress

B change

C input

D submit

💬 Discuss with your neighbor — then we'll reveal the answer!

✅ ANSWER

Which event fires continuously AS the user types in an input field?

A keypress

B change

C input

D submit

The input event fires on every keystroke, enabling real-time feedback. change only fires when the field loses focus after the value changed.

Regex Patterns in JS

```
// Test a regex against a value
const SID_REGEX = /^\d{9}$/;
const EMAIL_REGEX =
  /^[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$/;
const KR_PHONE_REGEX = /^01[016-9]\d{7,8}$/;

function validateStudentID(value) {
  if (!SID_REGEX.test(value)) {
    return "Student ID: 9 digits only";
  }
  return "";
}

// .test() returns true/false
console.log(SID_REGEX.test("123456789")); // true
console.log(SID_REGEX.test("12345678"));  // false
console.log(SID_REGEX.test("12345678a")); // false
```

Common validation regexes

`/^\d{9}$/`

Korean 9-digit student ID

`/^[^@]+@[^@]+$/`

Simple email format check

`/^01[016-9]\d{7,8}$/`

Korean mobile number

`/^(?=.*[A-Z])/`

At least 1 uppercase letter

`/^.{8,}$/`

Minimum 8 characters

QUICK QUIZ

The pattern /^\d{9}$/ validates a student ID. Which value PASSES this check?

A 12345678 (8 digits)

B 202312345 (9 digits)

C 20231234a (has a letter)

D 2023123456 (10 digits)

Discuss with your neighbor — then we'll reveal the answer!

✅ ANSWER

The pattern /^\d{9}$/ validates a student ID. Which value PASSES this check?

A 12345678 (8 digits)

B 202312345 (9 digits)

C 20231234a (has a letter)

D 2023123456 (10 digits)

The regex requires exactly 9 digits: ^ = start, \d{9} = nine digits, $ = end. 202312345 has exactly 9 digits and passes. All others have wrong length or non-digit characters.

Password Confirmation

```
<!-- HTML -->
<input type="password" id="pw"
       placeholder="Password">
<input type="password" id="pw2"
       placeholder="Confirm password">
<span id="pw2-error" class="error">
</span>

// JS
const pw  = document.getElementById("pw");
const pw2 = document.getElementById("pw2");
const pw2Err =
  document.getElementById("pw2-error");

pw2.addEventListener("input", () => {
  if (pw.value !== pw2.value) {
    pw2.setCustomValidity(
      "Passwords do not match"
    );
    pw2Err.textContent = "Passwords do not match";
  } else {
    pw2.setCustomValidity("");
    pw2Err.textContent = "";
  }
});
```

Comparing two fields

`Two inputs`

password + confirmPassword

`.value compare`

Strict equality ===

`On "input"`

Validate confirm on every key

`setCustomValidity`

Mark confirm as invalid in API

ACCESSIBLE VALIDATION

Connect errors to inputs for screen readers

Making validation work for everyone

Accessibility: aria-describedby

```
<!-- HTML: link input to its error span -->
<label for="email">Email</label>
<input
  type="email"
  id="email"
  aria-describedby="email-error"
  aria-invalid="false">
<span
  id="email-error"
  class="error"
  aria-live="polite"
></span>

// JS: update aria-invalid
function showError(input, errorEl, msg) {
  errorEl.textContent = msg;
  input.setAttribute("aria-invalid",
    msg ? "true" : "false");
}
```

Link inputs to error spans

`aria-describedby`

Points to id of error

`aria-invalid`

"true" when input has error

`aria-live="polite"`

Screen reader reads new text

`role="alert"`

Immediate announcement

The novalidate Attribute

✅ Browser bubbles are suppressed

novalidate disables the browser's built-in tooltips and prevents default submit-block behavior.

✅ checkValidity() still works

The validity API still runs — novalidate only stops the VISUAL popup, not the underlying validity state.

✅ Full styling control

Your custom errors follow your design system's colors, fonts and animations.

```
<!-- ADD novalidate to your form -->
<form id="reg-form" novalidate>
  <!-- inputs here -->
</form>
```

Without novalidate:

❌ Browser popup fires first
❌ Your JS error also shows
❌ User sees duplicate messages

With novalidate:

✅ Browser stays silent
✅ Only your styled error appears

Summary & Homework

`preventDefault()`

Stop submit; validate first

`checkValidity()`

Test a field programmatically

`setCustomValidity()`

Custom error + clear with ""

`validity object`

valueMissing, patternMismatch…

`"input" event`

Real-time feedback as user types

`aria-describedby`

Link input to error span for AT

HOMEWORK

Add full JS validation to your Monday registration form: use preventDefault, show inline error messages on "input" events, validate the student ID with /^\d{9}$/, and add a password-confirmation field. Add aria-describedby to every error span.

https://muxiddin19.github.io/WebPro/week7/WebPro_Week6_Fri_StudyGuide.html

https://muxiddin19.github.io/WebPro/week7/WebPro_Week6_Fri_FormValidation_v2.pptx

https://muxiddin19.github.io/WebPro/week7/WebPro_Week6_Fri_FormValidation_v2.pdf